Offline Camera Quality Validation Engine

OpenCV + Android CameraX for a waste-management app

Based on your waste-management use case, I recommend building a **fully offline, deterministic Camera Quality Validation Engine using OpenCV + Android CameraX**, without depending on paid AI services, cloud inference, or TensorFlow Lite.

This is a good approach for rugged Android enterprise devices.

Your application has a more demanding requirement than ordinary camera capture:

- Photograph waste containers and bins.
- Capture waste collection activities.
- Capture people involved in collection.
- Capture trucks and vehicle registration/inspection details.
- Capture field evidence before and after collection.
- Ensure the captured evidence is clear enough for audit and verification.
- Record the capture location in the metadata of every image.

I would design this as a reusable Android module with two independent responsibilities:

1. **Image Quality Validation:** Is this photograph technically good enough?
2. **Evidence Validation:** Does this photograph contain the required subject and show it adequately?

These two responsibilities should not be confused. A photograph can be perfectly sharp and well-lit but still fail because the truck or container is not fully visible.

1. Recommended technology stack

OpenCV — *Primary image processing engine*
Blur, exposure, glare, edges, contours, obstruction heuristics and image analysis.

Android CameraX — *Camera framework*
Preview, ImageAnalysis, ImageCapture, autofocus, exposure and zoom controls.

AndroidX ExifInterface
Image orientation, dimensions, timestamps and EXIF metadata.

Kotlin + Coroutines
Image processing, asynchronous analysis, concurrency and lifecycle management.

OpenCV is an established open-source computer vision library. Its current 4.x releases use the Apache 2.0 license. Review the exact version and bundled third-party components before distributing your application.

> **My recommendation:** Don't introduce TensorFlow Lite, ML Kit, cloud AI or another ML inference dependency in the initial implementation. Start with traditional computer vision and deterministic algorithms.

2. Can all six validations be implemented without AI/ML?

Yes, but there are some important limitations, particularly for identifying people, trucks and containers.

Here is the implementation strategy I recommend.

Requirement	OpenCV technique	ML required?
1. Sharpness / blur	Laplacian variance, Sobel gradients	No
2. Brightness / exposure	Grayscale luminance, histogram, clipping percentage	No
3. Glare / reflection	HSV analysis, highlight clipping, connected regions	No
4. Object positioning / framing	Contours, edges, geometry, guide overlays	No, with limitations
5. Resolution / integrity	Image dimensions, decode, EXIF validation	No
6. Obstruction detection	Edge density, local contrast, region analysis	No, with limitations

Important distinction: Object detection versus object positioning

Without an ML model, OpenCV cannot reliably understand that a particular object is a truck, a person or a waste container in arbitrary photographs.

However, we can still implement useful deterministic validation:

- Check whether the subject is inside a predefined capture guide.
- Detect whether the subject is cut off at image boundaries.
- Check whether the expected region contains sufficient edges and visual detail.
- Detect unusually large foreground obstructions using image-region analysis.
- Use manually selected capture profiles for containers, people, trucks and general evidence.

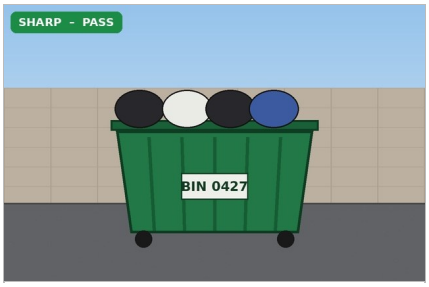
For example, when capturing a truck, display a truck-shaped framing guide. The user positions the truck inside the guide, and OpenCV checks image coverage and visibility.

This is much more predictable than pretending traditional computer vision can perform reliable semantic object detection.

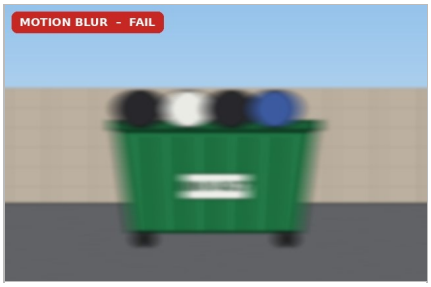
For people, we should not claim that a photograph contains a complete, unobstructed person based only on edge detection. That requires either a person detector or human confirmation.

3. Detailed implementation of the six validators

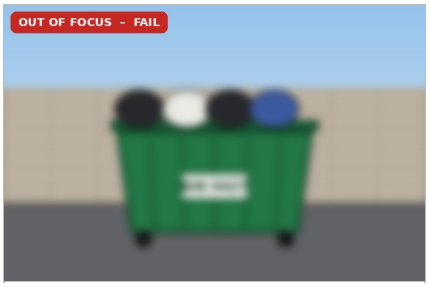
1. Sharpness / blur detection



Sharp image



Motion blur



Out of focus

Recommended algorithms:

- Variance of Laplacian.
- Sobel gradient magnitude.
- Edge density.

OpenCV operation:

kotlin

```
Imgproc.cvtColor(
    src, gray, Imgproc.COLOR_BGR2GRAY
)

Imgproc.Laplacian(
    gray, laplacian, CvType.CV_64F
)

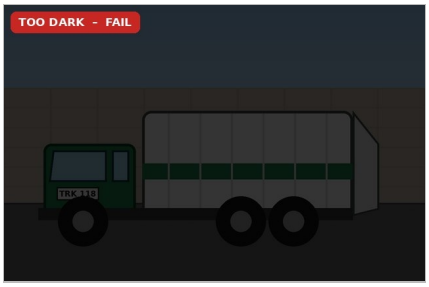
val mean = MatOfDouble()
val stddev = MatOfDouble()

Core.meanStdDev(laplacian, mean, stddev)

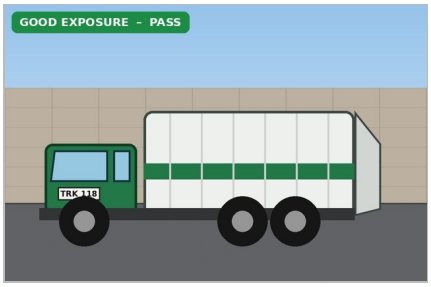
val variance = stddev.toArray()[0].pow(2)
```

> **Important:** Calibrate thresholds using real images from different tablet cameras. Analyze the relevant subject region as well as the whole image.

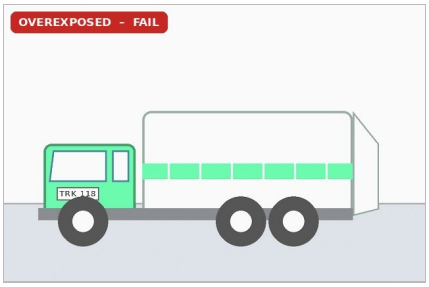
2. Brightness / exposure validation



Under-exposed



Correct exposure



Over-exposed

Recommended algorithms:

- Average luminance.
- Histogram distribution.
- Percentage of clipped dark pixels.
- Percentage of clipped bright pixels.

Example:

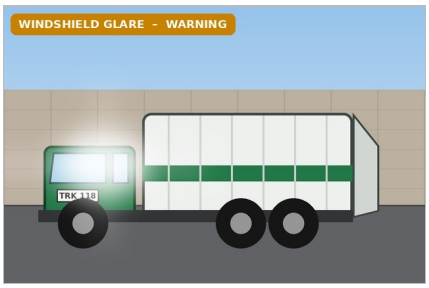
kotlin

```
Imgproc.cvtColor(
    src, gray, Imgproc.COLOR_BGR2GRAY
)

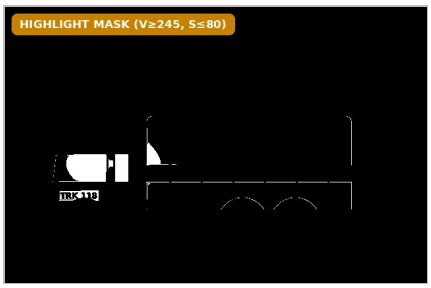
val meanBrightness = Core.mean(gray).`val`[0]
```

Don't rely only on average brightness. An image can have a normal average while its subject is completely dark.

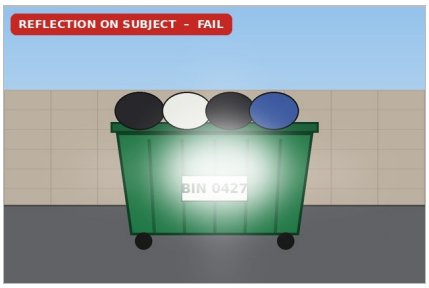
3. Glare / reflection detection



Glare on windshield



Detected highlight mask



Reflection hiding the subject

Recommended algorithms:

- HSV value-channel analysis.
- Highlight clipping percentage.
- Connected bright-region detection.
- Spatial distribution of overexposed pixels.

Example:

kotlin

```
Imgproc.cvtColor(
    src, hsv, Imgproc.COLOR_BGR2HSV
)

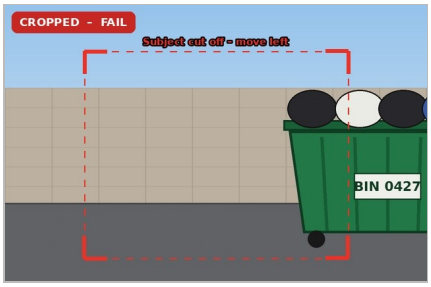
Core.inRange(
    hsv,
    Scalar(0.0, 0.0, 245.0),
    Scalar(180.0, 80.0, 255.0),
    highlightMask
)
```

This identifies bright, low-saturation regions. It is a glare indicator, not definitive proof of reflection.

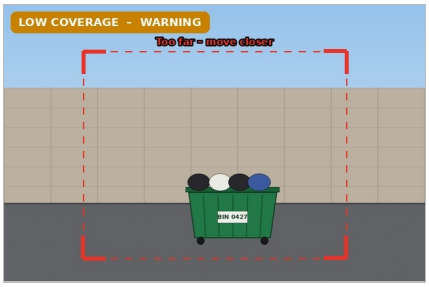
4. Object positioning / framing



Subject inside guide



Subject cut off at edge



Subject too small

Recommended algorithms:

- Contour detection.
- Edge detection using Canny.
- Bounding rectangle calculations.
- Image-region coverage.
- Subject boundary checks.

Example:

kotlin

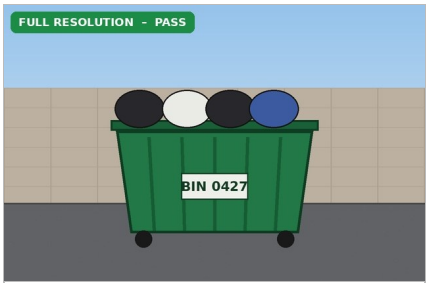
```
Imgproc.Canny(
    gray,
    edges,
    80.0,
    160.0
)

Imgproc.findContours(
    edges,
    contours,
    hierarchy,
    Imgproc.RETR_EXTERNAL,
    Imgproc.CHAIN_APPROX_SIMPLE
)
```

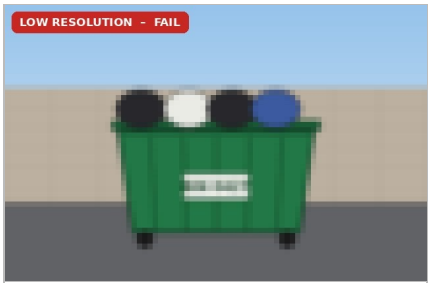
Use capture-specific framing overlays and compare the detected geometry against the configured region.

> **Limitation:** A contour is not necessarily the intended object. Complex backgrounds can produce incorrect detections.

5. Resolution / image integrity



Full-resolution capture



Down-scaled / low resolution

This is the most deterministic validator.

Validate:

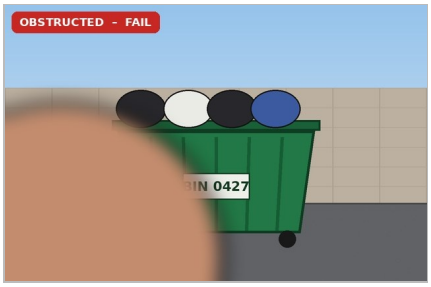
- Actual JPEG width and height.
- File size.
- Successful decoding.
- EXIF orientation.
- Image corruption.
- Minimum required dimensions.
- Unexpected downscaling.

Run these checks against the final captured JPEG, not just the preview frame.

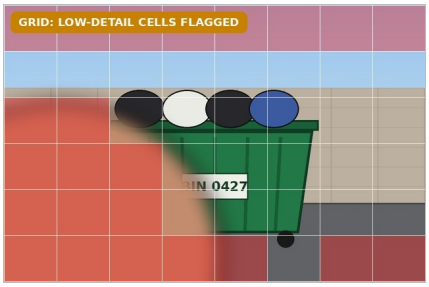
6. Obstruction detection



Unobstructed view



Finger over the lens



Grid analysis of blocked regions

Recommended heuristics:

- Local edge density.
- Local contrast.
- Large uniform regions.
- Unexpected foreground regions.
- Missing detail in required capture zones.

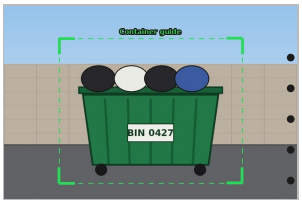
Divide the image into a grid and analyze individual regions.

This can detect suspiciously blocked regions, but cannot reliably distinguish a person's hand, a tree, a shadow or a naturally smooth surface without semantic recognition.

4. Build capture profiles for your waste-management workflow

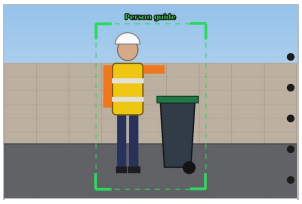
I strongly recommend not having one universal quality configuration.

Different photographs require different validation rules.



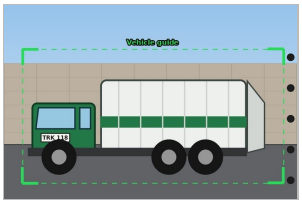
Profile A: Waste container

- Full container visible.
- Container edges reasonably sharp.
- Waste contents visible where required.
- No major obstruction.
- Adequate lighting.



Profile B: Person / worker

- Person positioned inside framing guide.
- Required body area visible.
- Sufficient facial or clothing detail where relevant.
- No major obstruction.
- No excessive motion blur.



Profile C: Truck / vehicle

- Vehicle positioned inside guide.
- Required vehicle area visible.
- Registration plate readable if required.
- No excessive windshield glare.
- Sufficient image resolution.



Profile D: General field evidence

- Overall image sharpness.
- Correct exposure.
- No major obstruction.
- Sufficient resolution.
- Image integrity.

Each profile should have its own configuration, thresholds, required regions and acceptance rules.

For example:

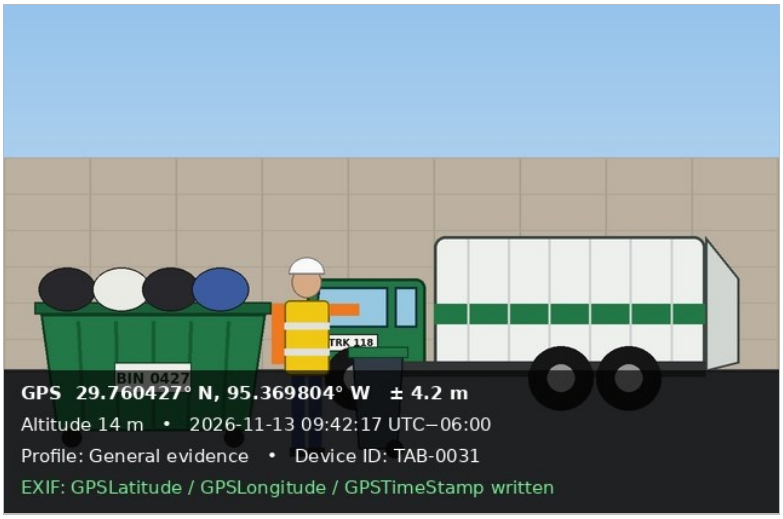
kotlin

```
data class CaptureProfile(
    val profileId: String,
    val minWidth: Int,
    val minHeight: Int,
    val blurThreshold: Double,
    val minBrightness: Double,
    val maxBrightness: Double,
    val maxGlarePercentage: Float,
    val minSubjectCoverage: Float,
    val requireLocation: Boolean,
    val maxLocationAccuracyMeters: Float,
    val maxLocationAgeSeconds: Int,
    val requiredChecks: Set<QualityCheck>
)
```

Keep these thresholds configurable rather than hardcoding them into OpenCV algorithms.

5. Location metadata in every captured image (mandatory)

Every evidence photograph must carry the location where it was taken, written into the image file itself, so the proof of location travels with the image and can be verified during audit even when the image is viewed outside the application.



Captured evidence with location metadata

What to record

EXIF tag	Content
GPSLatitude / GPSLatitudeRef	Latitude and N/S hemisphere
GPSLongitude / GPSLongitudeRef	Longitude and E/W hemisphere
GPSAltitude / GPSAltitudeRef	Altitude, when available
GPSTimeStamp / GPSDateStamp	UTC time and date of the location fix
GPSHPositioningError	Horizontal accuracy of the fix in metres
GPSProcessingMethod	Provider used (GPS, fused, network)
DateTimeOriginal / OffsetTimeOriginal	Local capture time and UTC offset

How to implement it

- Request `ACCESS_FINE_LOCATION` before opening the camera screen and keep a fresh location fix while the preview is visible.
- Use the device GPS through the fused location provider or `LocationManager`. GPS positioning works without mobile data, so this stays fully offline.
- Pass the fix to CameraX at capture time; CameraX writes the GPS EXIF tags into the JPEG.
- After capture, re-open the saved file with `ExifInterface`, confirm the GPS tags are present, and add the accuracy tag.
- Store the same latitude, longitude, accuracy and timestamp in the capture record in Room, so the location can be searched and reported without opening the image.

kotlin

```
val metadata = ImageCapture.Metadata().apply {
    location = lastKnownFix          // android.location.Location
}

val options = ImageCapture.OutputFileOptions
    .Builder(photoFile)
    .setMetadata(metadata)
    .build()

imageCapture.takePicture(options, executor, callback)

// Post-capture verification
val exif = ExifInterface(photoFile)
val latLong = exif.latLong            // null when GPS tags are missing
exif.setAttribute(
    ExifInterface.TAG_GPS_H_POSITIONING_ERROR,
    lastKnownFix.accuracy.toString()
)
exif.saveAttributes()
```

Validation rules

Condition	Result
No location permission or GPS switched off	Block capture and ask the user to enable location
No location fix yet	Show "Waiting for location" and keep capture disabled
Fix older than the configured age (for example 60 seconds)	Warning; refresh the fix before capture
Accuracy worse than the configured limit (for example 50 m)	Warning or fail, depending on the capture profile
GPS tags missing from the saved JPEG	Fail; retake the photograph
Mock-location provider detected	Flag the capture for review

> **Important:** Any step that re-encodes the image (resize, rotate, compress before upload) drops EXIF data unless it is copied across. Copy the GPS and timestamp tags to the processed file and verify them again before upload.

6. Live validation and post-capture validation

I recommend two different processing pipelines.

Live validation — Fast processing

Use lower-resolution frames, for example 640×480 or 1280×720 depending on device capability.

- Blur estimation.
- Brightness.
- Glare.
- Framing guidance.
- Basic obstruction indicators.

User feedback:

Message	Guidance
Too blurry	Hold tablet steady
Too dark	Improve lighting
Move back	Fit complete object
Good quality	Ready to capture

Post-capture validation — More detailed processing

- Analyze original captured JPEG.
- Full-resolution blur analysis.
- Detailed exposure and glare analysis.
- Image integrity and EXIF validation.
- Profile-specific framing checks.
- Location metadata present, recent and accurate enough.
- Final quality score.

Only accept the image after the final validation succeeds. If it fails, explain the exact reason and allow retaking.

One important implementation detail: use `ImageAnalysis.STRATEGY_KEEP_ONLY_LATEST` for live frames. Don't process every camera frame if the analyzer cannot keep up. Close every `ImageProxy` reliably, including error paths.

7. Cross-device compatibility strategy

Because the application must run on devices from several manufacturers, device-specific calibration is essential.

I would maintain a camera capability profile:

kotlin

```kotlin
data class CameraDeviceProfile(
    val manufacturer: String,
    val model: String,
    val cameraId: String,
    val supportedResolutions: List<Resolution>,
    val hasAutoFocus: Boolean,
    val hasFlash: Boolean,
    val minFocusDistance: Float?,
    val supportedHardwareLevel: Int
)
```

Use CameraX/Camera2 characteristics to discover capabilities rather than relying on model-name checks.

Do not assume that all devices have autofocus. Some may have fixed-focus cameras.

Also, CameraX can select different actual stream resolutions depending on the device and the combination of use cases. Inspect the actual output dimensions.

8. Proposed modular architecture

For your Kotlin + Compose + Hilt multi-module application, I would use:

text

```text
:core:camera
    CameraXManager
    CameraPreview
    CameraCaptureManager
    CameraCapabilityProvider
    CameraFocusController

:core:opencv
    OpenCvInitializer
    MatConverter
    OpenCvImageProcessor

:core:image-quality
    BlurValidator
    ExposureValidator
    GlareValidator
    FramingValidator
    ResolutionValidator
    ObstructionValidator
    QualityScoreCalculator

:core:location
    LocationProvider
    LocationFixValidator
    ExifLocationWriter

:core:capture-profile
    CaptureProfile
    CaptureProfileProvider
    ProfileThresholds

:feature:camera
    CameraScreen
    CameraViewModel
    CameraUiState
    CameraUiEvent

:feature:image-review
    ImageReviewScreen
    QualityReportScreen

:core:data
    CaptureRepository
    CaptureMetadata
    CaptureLocalDataSource
```

Use separate executors or dispatchers for camera analysis and post-capture processing. Avoid converting every preview frame into a large bitmap unnecessarily; work with YUV planes where practical and convert only when needed.

9. Suggested POC acceptance criteria

Before integrating into the waste-management application, I would build a standalone camera POC and test it on actual tablets.

Test	Expected result
Clear image	Pass
Intentionally blurred image	Fail
Low-light image	Warning/fail
Overexposed image	Warning/fail
Strong reflection	Warning
Partially cropped container	Framing warning
Blocked camera/subject area	Obstruction warning
Low-resolution JPEG	Fail
Corrupted image	Fail
Portrait/landscape capture	Correct orientation
Capture with a valid location fix	GPS tags present in the JPEG
Capture with location off or no fix	Capture blocked
Different tablet models	Consistent calibrated decisions

Create a reference dataset with good and bad images from each target tablet. Manually label them and measure false acceptance and false rejection rates.

OpenCV version and distribution

OpenCV's official Android AAR distribution is available through Maven Central from version 4.9.0 onwards.

I would:

- Use the official OpenCV Android package.
- Pin the exact version.
- Review its Apache 2.0 license and third-party notices.
- Test the native libraries against your supported ABIs.
- Check Android 16 KB memory-page compatibility before deployment. OpenCV's latest 5.0.0 release specifically documents an Android SDK binary compatibility consideration.

10. My final recommendation

Recommended approach
Start with this combination:

CameraX + OpenCV + Kotlin

100% offline

For the six proposed quality checks and location tagging; no external AI inference required.

- CameraX handles camera hardware and capture.
- OpenCV handles image quality algorithms.
- Kotlin handles validation rules and scoring.
- Compose handles camera UI and guidance.
- The device GPS supplies the location written into each image's EXIF metadata.
- Room stores capture metadata, location and quality reports.
- WorkManager handles eventual upload when connectivity is available.

One architectural decision I would make from the beginning: **never make image quality dependent on a specific tablet manufacturer.** Keep the algorithms common, and calibrate thresholds using device-specific test results and

capture profiles.

11. One decision before building the POC

For the object positioning and obstruction requirements, there are two possible approaches without AI/ML:

- **Approach A – Guided capture:** The application displays different framing overlays for container, person, truck and vehicle. OpenCV validates image quality and geometric coverage.
- **Approach B – Free capture:** Users can photograph anything, and OpenCV attempts to identify subject boundaries and obstructions using traditional computer vision. This is less reliable for arbitrary scenes.

I recommend Approach A for your waste collection application.

Would you prefer to use **guided capture with separate overlays for containers, people, trucks and vehicles**, while retaining a free-capture mode for general field evidence?